

UNIVERSIDADE ESTADUAL DO PIAUÍ - UESPI
CAMPUS DRª JOSEFINA DEMES
CURSO BACHARELADO EM CIÊNCIA DA
COMPUTAÇÃO
DISCIPLINA: PADRÕES DE PROJETO

Estudo de Caso 3 — Builder

Sistema de Geração de Relatórios Personalizados
Alunos: Hilário, Ryan e Luca

1. Análise do problema

a) Qual é o principal problema relacionado à criação dos objetos Relatorio? O objeto Relatorio é um objeto complexo: possui cerca de doze atributos, dos quais apenas três são obrigatórios (título, período e autor) e todos os demais são opcionais. Além disso, há regras de consistência entre os atributos: pelo menos um gráfico ou tabela; impressão exige cabeçalho e rodapé; numeração só com mais de uma página; formato PDF, HTML ou DOCX, com PDF como padrão. O problema central é que, sem um mecanismo próprio de montagem, o código cliente precisa conhecer todos esses detalhes e combinar os parâmetros corretamente, e nada garante que o objeto nasça em um estado válido. As regras tendem a ficar duplicadas em vários pontos do sistema.

b) Por que um construtor com muitos parâmetros dificulta a leitura e a manutenção?

• **Ilegibilidade:** uma chamada como new

Relatorio("Vendas","Set/26","Ana",null,tabela,null,null,true,false,null)
) não informa o significado de cada argumento; é preciso consultar a assinatura.

- **Erros silenciosos de ordem:** título, período e autor são todos String; trocá-los não gera erro de compilação.
- **Excesso de null:** cada opcional não usado precisa ser preenchido com null ou valor neutro.
- **Explosão de sobrecargas** (construtor telescópico): para cobrir as combinações de opcionais, surgem várias versões do construtor.
- **Manutenção frágil:** incluir um novo atributo altera a assinatura e quebra todas as chamadas existentes.
- **Sem ponto claro de validação** das regras entre atributos.

c) Por que apenas usar vários setters não resolve todos os problemas?

Os setters melhoram a legibilidade (cada valor é nomeado), mas deixam outros problemas:

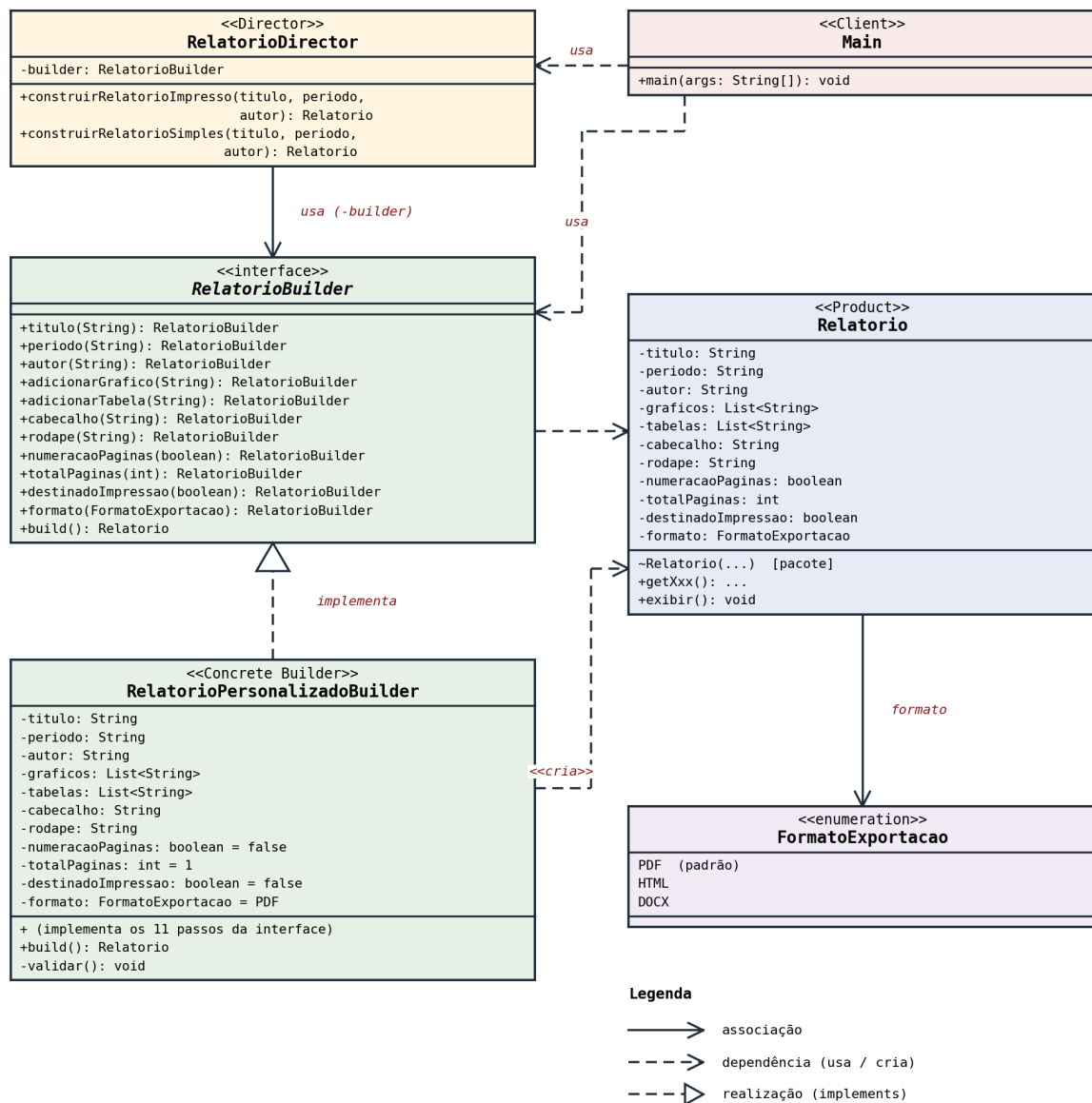
- **Estado intermediário inválido:** entre o new e o último setter o objeto existe incompleto, por exemplo sem autor ou sem conteúdo.
- **Mutabilidade:** o objeto continua podendo ser alterado depois de pronto, o que quebra as regras já verificadas e impede torná-lo imutável (e seguro para uso concorrente).
- **Nenhuma garantia de obrigatórios:** nada impede o cliente de esquecer de chamar um setter obrigatório.
- **Validação dispersa:** como não há momento em que o objeto é declarado "pronto", as regras entre atributos (impressão, numeração, conteúdo mínimo) não têm onde ser verificadas de forma centralizada.

- **Dependência de ordem:** certas regras (numeração x número de páginas) dependem de a configuração está completa, o que um setter isolado não sabe.

2. Identificação dos elementos do padrão

Elemento	Classe na solução	Responsabilidade
Product	Relatorio	Objeto complexo que se deseja obter. É imutável (atributos <i>final</i>) e seu construtor tem visibilidade de pacote, de modo que só o Builder consegue instanciá-lo. Guarda título, período, autor, gráficos, tabelas, cabeçalho, rodapé, numeração, total de páginas, indicação de impressão e formato.
Builder	RelatorioBuilder	Interface abstrata que declara os passos de construção (titulo, periodo, autor, adicionarGrafico, adicionarTabela, cabecalho, rodape, numeracaoPaginas, totalPaginas, destinadoImpressao, formato) e o método <code>build()</code> . Os passos retornam o próprio Builder (interface fluente).
Concrete Builder	RelatorioPersonalizadoBuilder	Implementa os passos: acumula o estado parcial, define os valores padrão (formato = PDF, 1 página, sem numeração, sem impressão), executa as validações em <code>build()</code> e só então cria o Relatorio final.
Director	RelatorioDirector	Encapsula receitas prontas de construção, isto é, a sequência e a combinação de passos usadas com frequência: <code>construirRelatorioImpresso()</code> (completo, com cabeçalho, rodapé, numeração e DOCX) e <code>construirRelatorioSimples()</code> (apenas o mínimo). Recebe um RelatorioBuilder e depende apenas da interface.
Client	Main	Código que usa o padrão. Ora chama o Builder diretamente, configurando só o que precisa, ora pede ao Director uma receita pronta. Não conhece a estrutura interna nem o construtor de Relatorio.

3. Diagrama UML de classes



Como o relatório é construído até chegar ao objeto final:

- 1. O Client (Main) cria um RelatorioPersonalizadoBuilder e, se quiser uma receita pronta, o entrega a um RelatorioDirector.
- 2. Os passos são chamados em sequência (diretamente pelo Client ou pelo Director): cada chamada grava um dado no estado interno do Concrete Builder e devolve o próprio Builder.

- 3. Quando a configuração termina, chama-se `build()`.
- 4. Dentro de `build()`, o método privado `validar()` verifica as regras. Se alguma falhar, é lançada `IllegalStateException` e nenhum objeto é criado.
- 5. Se tudo estiver correto, o `Concrete Builder` instancia o `Relatorio (Product)`, imutável, usando o valor padrão `PDF` quando o formato não foi informado.

4. Implementação em Java

A solução foi compilada e executada com Java 21; a saída está na seção 5. Cada classe fica em seu próprio arquivo (pacote padrão). Para executar: `javac *.java` e depois `java Main`.

Requisito	Como foi atendido
A) Obrigatórios	título, período e autor (validados como não vazios em <code>validar()</code>).
B) Opcionais (mín. 4)	gráficos, tabelas, cabeçalho, rodapé, numeração de páginas, total de páginas, destinado à impressão e formato (oito ao todo).
C) Valores padrão (mín. 1)	<code>formato = PDF</code> ; <code>totalPaginas = 1</code> ; <code>numeracaoPaginas = false</code> ; <code>destinadoImpressao = false</code> .
D) Validações (mín. 3)	(1) título/período/autor obrigatórios; (2) pelo menos um gráfico ou uma tabela; (3) impressão exige cabeçalho e rodapé; (4) numeração só com mais de uma página.

Link da implementação github:

https://github.com/HilarioMarques/EstudoCaso_Builder/tree/main

5. Demonstração

```
hilario@hilario-Inspiron-15-3520:~/Documentos/Uespi-CiênciaDaComput
r/Implementacao$ java Main
=====
[Cabeçalho] Setor Financeiro
-----
Título: Relatório Financeiro Mensal
Período : Setembro/2026
Autor: Ana Souza
Gráficos : [Receita x Despesa, Fluxo de caixa]
Tabelas : [Balancete]
Formato : PDF
Páginas : 5 (com numeração)
-----
[Rodapé] Confidencial
=====

>>> Relatório 2: Vendas (mínimo, HTML)
=====
Título: Vendas da Semana
Período : 23/09 a 29/09/2026
Autor: Carlos Lima
Gráficos : -
Tabelas : [Vendas por produto]
Formato : HTML
Páginas : 1 (sem numeração)
=====
```

```

>>> Relatório 3: Corporativo (Director, impressão, DOCX)
=====
[Cabeçalho] Relatório Corporativo
-----
Título: Relatório Anual de Atividades
Período : 2026
Autor: Diretoria de Operações
Gráficos : [Desempenho por setor]
Tabelas : [Resumo de indicadores]
Formato : DOCX (destinado à impressão)
Páginas : 8 (com numeração)
-----
[Rodapé] Documento interno - uso restrito
=====

>>> Tentativa inválida 1: sem gráfico e sem tabela
Construção bloqueada: O relatório deve possuir pelo menos um gráfico ou uma tabela.

>>> Tentativa inválida 2: impressão sem cabeçalho e rodapé
Construção bloqueada: Relatórios destinados à impressão devem possuir cabeçalho e rodapé.

>>> Tentativa inválida 3: numeração em relatório de 1 página
Construção bloqueada: A numeração de páginas só pode ser usada em relatórios com mais de uma página.
hilario@hilario-Inspiron-15-3520:~/Documentos/Uespi-CiênciaDaComputacao/BlocoV/PadroesProjetos/Estudo
r/Implementacao$

```

6. Análise da solução

a) Como o Builder contribuiu para melhorar a legibilidade da criação dos relatórios?

Cada valor é atribuído por um método com nome próprio (.titulo("..."), .adicionarTabela(...), formato(HTML)), de modo que a chamada se lê quase como uma frase e dispensa consultar a ordem dos parâmetros. O cliente informa apenas o que interessa, sem passar null para o que não usa, e os valores padrão são aplicados automaticamente.

b) Qual é a vantagem de separar a construção do Relatório da representação final do objeto?

A classe Relatório fica simples, imutável e sempre válida, pois só representa o resultado; a complexidade de montagem, os valores padrão e as validações ficam no Builder. Isso dá coesão a cada classe (responsabilidade única) e permite mudar o modo de construir (novas regras, novas receitas, outro

Concrete Builder) sem alterar o produto nem os clientes. O mesmo processo de construção pode ainda gerar representações diferentes

c) Qual é a responsabilidade do método build()?

É o ponto único de finalização: executa as validações sobre o estado acumulado (obrigatórios, conteúdo mínimo, impressão, numeração) e, só se todas passarem, cria e devolve o Relatório imutável, com cópias defensivas das listas. Se alguma regra falhar, lança exceção e impede a existência de um objeto inválido.

d) Qual é a diferença entre a responsabilidade do Builder e a do Concrete Builder?

O Builder (RelatorioBuilder) é o contrato: define quais passos existem e o que build() retorna, sem dizer como são executados. O Concrete Builder

(RelatorioPersonalizadoBuilder) é a implementação: guarda o estado, define os valores padrão, aplica as validações e instancia o produto. Por isso o Director e o Client dependem da interface, e outra variação de construção poderia ser trocada sem alterar quem a usa.

e) Que tipo de alteração no sistema seria facilitada pelo uso do Builder?

- **Novo atributo opcional** (por exemplo, marca d'água ou idioma): adiciona-se um passo no Builder, sem alterar as chamadas já existentes.

- **Nova regra de validação:** basta incluí-la em validar(), em um único lugar.

- **Novo formato de exportação:** acrescenta-se ao enum `FormatoExportacao`.
- **Novas configurações frequentes:** criam-se novas receitas no Director.
- **Outro tipo de relatório:** cria-se um novo Concrete Builder que implementa a mesma interface.

f) Quais são as desvantagens de utilizar o padrão Builder nesse cenário?

- **Mais classes e código** (interface, Concrete Builder e Director, além do produto), o que pode ser excessivo se o relatório fosse simples.
- **Duplicação de atributos:** os campos aparecem no Builder e no Relatorio, e a inclusão de um novo atributo exige alterar ambos (e a interface).
- **Erros só em tempo de execução:** como a validação ocorre em `build()`, esquecer um obrigatório só é detectado ao executar, não na compilação (existem variações, como o Builder com etapas tipadas, que evitam isso, ao custo de mais complexidade).
- **Estado mutável no Builder:** reutilizar a mesma instância de Builder para vários relatórios pode carregar dados do anterior, sendo mais seguro criar um Builder novo a cada relatório.
- **Custo leve de indireção** para construir cada objeto, geralmente irrelevante neste cenário.